Lecture 09

Currying, FP Idioms

When Things Evaluate

- We know:
  - Function body not evaluated until call
  - Function body evaluated every time function is called
    - In environment from when function was defined
  - Variable bindings evaluate right-hand-side once at binding time
    - NOT every time variable is used
- w/ closures, can avoid repeat computations that don’t depend on args
  - Can make code run faster, but also interesting semantics

What happens when this code is evaluated?

```
let f x = List.hd []
let result = if false then f 1 else List.hd [1;2;3]
```

Recomputation

These both work and are equivalent

```
let all_shorter xs s =
  filter (fun x -> String.length x < String.length s) xs

let all_shorter' xs s =
  let n = String.length s in
  filter (fun x -> String.length x < n) xs
```

Recomputation

These both work and are equivalent

```
let all_shorter xs s =
  filter (fun x -> String.length x < String.length s) xs

let all_shorter' xs s =
  let n = String.length s in
  filter (fun x -> String.length x < n) xs
```

First version computes `String.length s` repeatedly (once per `x` in `xs`)
Second version computes `String.length s` once before filtering

- No new features! Just new use of closures :)

Map

```
(* ('a -> 'b) -> 'a list -> 'b list *)
let rec map f xs =
  match xs with
  | [] -> []
  | x :: xs' -> f x :: map f xs'
```

Hall-of-fame higher-order function

- Name is standard (used for any sort of collection data structure)
  - Generally provided in standard libraries for standard collections
- Used all the time in functional code, thus *idiomatic*
  - Using it where appropriate isn't just "shorter code" but *communicates what you are doing*
  - Conversely, not using it when "doing a map" confuses your reader

Filter

```
(* ('a -> bool) -> 'a list -> 'a list *)
let rec filter f xs =
  match xs with
  | [] -> []
  | x :: xs' -> if f x then
                  x :: filter f xs'
                else
                  filter f xs'
```

- Another hall-of-famer
  - Keep only some elements from a collection
  - Also an idiom, so you should use whenever “doing a filter”

Fold

```
let rec fold_left f acc xs =
 match xs with
 | [] -> acc
 | x :: xs' -> fold_left f (f acc x) xs'
```

- Another “hall-of-fame higher-order function” aka **`fold`** aka **`reduce`**
  - Accumulate answer by repeatedly applying **`f`** :
    - **`fold_left f acc [v1; v2; v3]`**
      → **`f (f (f (f acc v1) v2) v3)`**
  - **`fold_left:('a -> 'b -> 'a)-> 'a -> 'b list -> 'a`**
  - Contrast with **`fold_right`** (goes through list “the other way”)

Group Work

1. Write a function that sums up all of the even values in a list of integers
    - Hint: `x mod 2` will be true when `x` is even and false when `x` is odd
2. Write a function that takes a list of strings and returns a list containing the lengths of the strings in the input list that start with “a”
    - Hint: `String.starts_with ~prefix:"a" s` will return true if `s` starts with `"a"` (false otherwise)
3. Write a function that returns the max of the squares of the numbers in a list
    - Hint: This function should return 0 when the list is empty

```
(* ('a -> 'b) -> 'a list -> 'b list *)
let rec map f xs =
  match xs with
  | [] -> []
  | x :: xs' -> f x :: map f xs'

(* ('a -> bool) -> 'a list -> 'a list *)
let rec filter f xs =
  match xs with
  | [] -> []
  | x :: xs' -> if f x then
                  x :: filter f xs'
                else
                  filter f xs'

(* ('a -> 'b -> 'a) -> 'a -> 'b list *)
let rec fold_left f acc xs =
 match xs with
 | [] -> acc
 | x :: xs' -> fold_left f (f acc x) xs'
```

Iterators

- **`map`**, **`filter`**, **`fold`** are similar to iteration patterns we see in imperative PLs
  - But not “built into” OCaml -- we can just define them ourselves!
  - “Just an idiom”! Power from combining elegant features.
- Separate “traversal” from “computation”
  - Allows reusing traversal code
  - Allows reusing computation code
  - Allows common way to talk about traversing data structures
    - Most collections provide **`map`**, **`filter`**, **`fold`** functions that “work the same way”

A Point on Style

- **Bad:** `if e then true else false`
- **Good:** `e`
- **Bad:** `fun x -> f x`
- **Good:** `f`
- **Bad:** `n_times ((fun x -> List.tl x), 3, xs)`
- **Good:** `n_times (List.tl, 3, xs)`

Currying

- Recall every OCaml function takes exactly one argument
- Previously encoded *n* arguments via one *n*-tuple
- Another way: Take one argument and return a function that takes another argument and…
  - Called “currying” after logician Haskell Curry

Example

```
let sorted3 = fun x -> fun y -> fun z ->
                    z >= y && y >= x

let t = ((sorted3 7) 9) 11
```

- Calling `(sorted3 7)` returns a closure with:
  - Code `fun y -> fun z -> z >= y && y >= x`
  - Environment maps `x` to `7`
- Calling *that* closure with `9` returns a closure with:
  - Code `fun z -> z >= y && y >= x`
  - Environment maps `x` to `7`, `y` to `9`
- Calling *that* closure with `11` returns `true`

Syntactic sugar, part 1

```
let sorted3 = fun x -> fun y -> fun z ->
                  z >= y && y >= x

let t = ((sorted3 7) 9) 11
let t = sorted3 7 9 11
```

- In general, `e1 e2 e3 e4 …,` means `(…((e1 e2) e3) e4)`
- So instead of `((sorted3 7) 9) 11`, can write `sorted3 7 9 11`
- Callers can just think “multi-argument function with spaces instead of a tuple expression”
  - Different than tupling; caller and callee must use same technique

Syntactic sugar, part 2

```
let sorted3 = fun x -> fun y -> fun z ->
                  z >= y && y >= x

let sorted3 x y z = z >= y && y >= x
let t = sorted3 7 9 11
```

- In general, `let [rec] f p1 p2 p3 … = e`
  means `let [rec] f p1 = fun p2 -> fun p3 -> … -> e`
- So can just write `let sorted3 x y z = z >=y && y >= x`
- Callees can just think “multi-argument function with spaces instead of a tuple pattern”
  - Different than tupling; caller and callee must use same technique

Final version

```
let sorted3 x y z = z >= y && y >= x
let t = sorted3 7 9 11
```

As elegant syntactic sugar (even fewer characters than tupling) for:

```
let sorted3 = fun x -> fun y -> fun z ->
                  z >= y && y >= x

let t = ((sorted3 7) 9) 11
```

Unnecessary function wrapping

```
let rec fold_left f acc xs =
  match xs with
  | [] -> acc
  | x :: xs' -> fold_left f (f acc x) xs'

let sum_bad_style xs =
  fold_left (fun acc x -> acc + x) 0 xs
```

As we already know, `fold_left (fun acc x -> acc + x) 0` evaluates to a closure that given `xs`, evaluates the match-expression with `f` bound to `(fun acc x -> acc + x)` and `acc` bound to `0`

Unnecessary function wrapping

```
let rec fold_left f acc xs =
  match xs with
  | [] -> acc
  | x :: xs' -> fold_left f (f acc x) xs'

let sum_bad_style xs =
  fold_left (fun acc x -> acc + x) 0 xs
let sum_good_style =
  fold_left (+) 0
```

- Previously learned not to write `let f x = g x`
- This is the same thing with `fold_left (fun acc x -> acc + x) 0` in place of `g`

“Too Few Arguments”

- Previously used currying to simulate multiple arguments

- But if caller provides “too few” arguments, we get back a closure “waiting for the remaining arguments”
  - Called *partial application*
  - Convenient and useful
  - Can be done with *any* curried function

- No new semantics here: a pleasant, general idiom

Iterators redux

Partial application is particularly nice for iterators

- Provided iterator implementations “put the function argument first”
- So that’s what libraries do

```
let remove_negs_meh xs = List.filter (fun x -> x >= 0) xs

let remove_negs = List.filter (fun x -> x >= 0)

let remove_all n = List.filter (fun x -> x <> n)

let remove_zeros = remove_all 0
```

In fact (!)

- General style in OCaml is currying for multiple arguments
  - Not tupling
  - Expect to see curried arguments in libraries

```
let rec append xs ys = (* 'a list -> 'a list -> 'a list *)
  match xs with
  | [] -> ys
  | x::xs' -> x :: append xs' ys
```

- It’s “weird” that we used tupling for 3 weeks, but wanted to show currying after we could understand its semantics

Efficiency / convenience

So which is faster/better: tupling or currying multiple-arguments?

- Both are constant-time operations, so it doesn't matter in most of your code – "plenty fast"
  - Don't program against an *implementation* until it matters!
- For the small (zero?) part where efficiency matters:
  - It turns out OCaml compiles currying more efficiently (it optimizes full application)
  - OCaml could have made the other implementation choice
- More interesting trade-off is convenience:
  - Partial application vs. "compute a tuple to pass"

The Value Restriction Appears ☹️

If you use partial application to *create a polymorphic function*, it may not work due to the value restriction

- May give it the monomorphic type you first use it with or a strange type error
- This should surprise you; you did nothing wrong ☺ but you still must change your code
- See the code for workaround: not-so-unnecessary function wrapping
- Can discuss a bit more when discussing type inference

Group Work

Write a function `process_option` that takes as input two functions, `f` and `g` (curried)

`f` has type `'a -> 'b option` and `g` has type `'b -> 'c option`

`process_option` should return a value of type `'a -> 'c option`